

Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente

2027-1

Práctica 01. Repaso de Java e Introducción a la programación Multihilos

ALUMNOs:

Martínez Hidalgo Paola Mildred - 319300217
Dominguez Sanchez Gustavo Alonso - 418047121

Fecha de entrega:
27 de agosto de 2026

Profesor:
Gilde Valeria Rodríguez



Ejercicios

1. Lee lo siguiente `https://www.evanjones.ca/software/threading-linus-msg.html` y comparte en máximo 4 líneas de computadora a que se refiere Linus Torvalds con un contexto de ejecución y cómo se relaciona con la definición en la sección 1 de esta práctica.

- Mildred: Torvalds dice que un contexto de ejecución (COE) es el conjunto total de estado de una unidad de ejecución (CPU, memoria, permisos, archivos, señales). Para él "hilo" y "proceso" no son entidades distintas, sino subconjuntos de qué estado se comparte entre COEs. La sección 1 sí los separa como definiciones fijas (proceso = programa en ejecución; hilo = proceso paralelo que comparte recursos), justo la distinción que Linus rechaza.
- Alonso: Aunque en la definición que se nos da en el capítulo se nos define por separado hilo y proceso, en la que incluso se nos brinda una arquitectura bien definida en cada uno, Linus cree ambos son subconjuntos de un contexto de ejecución (COE) y que incluso son triviales bajo su enfoque, sin embargo, parece ser más un paradigma personal, porque menciona que incluso si alguien lo desea, Linux podia trabajarse tanto con COE (principalmente) como con hilos y procesos utilizando otras bibliotecas. Por lo tanto, es claro que existe una relación inherente entre hilo y proceso, pero si puede ser manejado bajo conceptos distintos.

2. ¿Cuántos hilos tiene disponibles tu computadora? Ejecuta Runtime.getRuntime().availableProcessors(), si son más de uno en el equipo escriban el de cada uno.

- Mildred: Después de correr el comando dado se obtuvo como resultado; Hilos disponibles: 4
- Alonso: Hilos disponibles: 8

3. Revisa el programa Determinante concurrente y responde ¿Cuánto tiempo tarda en ejecutarse?

- Mildred: Como salida el programa me dio Program took 4492401ns, result: -18, una vez pasado a segundos es de 4.49 ms.
- Alonso: Salida: Program took 1380021ns, result: -18. Esto es 1.38 milisegundos.

4. El programa Determinante concurrente está implementado extendiendo la clase Thread. Implementa el programa utilizando la interfaz Runnable.

El programa se encuentra en el archivo DeterminanteConcurrenteRunnable.java. La forma de calcular el determinante es exactamente la misma que en el programa original, lo único que cambia es cómo creamos y manejamos los hilos. Antes, la clase misma era un hilo (heredaba de Thread), pero la clase ahora solo describe la tarea que se va a hacer (implementa Runnable), y luego esa tarea se le "entrega" a un hilo aparte para que la ejecute (new Thread(runnable)). Es un cambio más de organización que de lógica, el resultado es el mismo, solo cambia quién es responsable de qué.

5. Implementa el programa Determinante concurrente de forma secuencial. El programa se encuentra en el archivo DeterminanteSecuencial.java. Para una mejor comparación entre los tipos de cómputo, se quiso mantener el mismo modelado de objetos vistos en DeterminananteConcurrente para las matrices, e incluso la misma forma de calcular el determinante dividiendolo en resultados parciales. Sin embargo, en este programa secuencial fue removido la herencia/implementación de Threads/Runnable, y realizar los mismos pasos pero secuencialmente.

6. Implementa del programa Determinante concurrente para dos hilos (en vez de seis).

El programa se encuentra en el archivo DeterminanteConcurrente2Hilos.java. Para pasar de seis hilos a dos, lo que hicimos fue agrupar el trabajo: en vez de que cada hilo calcule un solo producto, ahora un hilo se encarga de calcular los tres productos que se suman (los positivos) y el otro hilo se encarga de los tres que se restan (los negativos). Al final simplemente restamos lo que dio cada hilo, y así llegamos al mismo resultado que con la versión original, solo que repartiendo el trabajo entre menos hilos.

7. Compara las 3 implementaciones: el programa Determinante concurrente para dos hilos, para seis hilos y el programa secuencial. Responde: ¿A qué se debe el orden en el que se ordenan los tiempos de ejecución de cada programa?
Estos son ejemplos de los tiempos de ejecución de cada programa:

Programa	Tiempo(ns)
Secuencial	9341
Concurrente 2 hilos	518,826
Concurrente 6 hilos	1,144,598

Es natural pensar que al hacer cálculos concurrentes, el tiempo de ejecución total sería menor, sin embargo, siempre es necesario considerar que hay costo computacional en el manejo de hilos. Esto es más notable en programas cuya entrada sea pequeña, el cual es nuestro caso, ya que para calcular el determinante de una matriz de 3x3 no es necesario dividir el trabajo en hilos. Sale más costoso crear e inicializar los hilos, realizar los cálculos de cálculos de cada hilo, administrar los `join()` y combinar los resultados, que simplemente realizar los pasos secuencialmente. En resumente, para este caso en particular, el cálculo matemático es relativamente más pequeño que el costo de la administración de hilos, evidenciando que la concurrencia no es la mejor opción en trabajos simples.

8. Si utilizas la Ley de Amdahl entre el programa Determinante concurrente para dos hilos y el programa secuencial. ¿El resultado es mayor o menor a 1? ¿Por qué?

Tomando en cuanto los tiempos de ejecución anteriores, de acuerdo a la Ley de Amdahl:

$$A = \frac{1}{(1-p) + \frac{p}{n}} = \frac{T_{antiguo}}{T_{mejorado}} = \frac{T_{secuencial}}{T_{concurrente}} = \frac{9341}{518,826} \approx 0{,}018 \tag{1}$$

Por lo que el resultado es mucho menor que 1. Tal como se argumentó en el inciso anterior, esto se debe al costo de la concurrencia en un problema con entradas pequeñas. De hecho, el algoritmo visto de manera inversa, el programa secuencial es 55.5x veces más rápido que el programa concurrente. En este sentido, la parte paralelizable (p) es pequeña que hace que A diste mucho del valor de 1.

9. Describe con tus propias palabras en máximo dos líneas para qué sirve el método join(). Si no utilizas el método join() en Determinante Concurrente, ¿sigue funcionando?

join() sirve para que un hilo espere a que otro termine antes de seguir adelante, por ejemplo, que el programa principal no continúe hasta que todos los hilos hayan acabado su trabajo. Si quitamos el join() en Determinante Concurrente, el programa no se rompe ni marca error, sigue corriendo normal. El problema es que ya no podemos confiar en el resultado, puede que el programa principal intente leer lo que calculó cada hilo antes de que ese hilo haya terminado, y en ese caso tomaría un valor vacío en vez del correcto. O sea, puede que a veces dé bien y a veces mal, dependiendo de qué tan rápido vaya cada hilo, y eso es justo lo que join() evita.